

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

## Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

А.С. Мышкевич

студент группы ПО-13

Проверил:

А.А. Крощенко

доц. кафедры ИИТ,

05.03.2026

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику.

При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Выполнение:

**Код программы**

**Crud.py**

```
"""CRUD операции с базой данных."""
from datetime import date
from typing import Optional

from sqlalchemy.orm import Session

import models
import schemas

# AUTHORS
def get_author(db: Session, author_id: int):
    """Получение автора по ID."""
    return db.query(models.Author).filter(models.Author.id == author_id).first()

def get_authors(db: Session, skip: int = 0, limit: int = 100):
    """Получение списка авторов."""
    return db.query(models.Author).offset(skip).limit(limit).all()

def create_author(db: Session, author: schemas.AuthorCreate):
    """Создание автора."""
    db_author = models.Author(**author.model_dump())
    db.add(db_author)
    db.commit()
```

```

db.refresh(db_author)
return db_author

def update_author(db: Session, author_id: int, author: schemas.AuthorCreate):
    """Обновление автора."""
    db_author = get_author(db, author_id)
    if db_author:
        for key, value in author.model_dump().items():
            setattr(db_author, key, value)
        db.commit()
        db.refresh(db_author)
    return db_author

def delete_author(db: Session, author_id: int):
    """Удаление автора."""
    db_author = get_author(db, author_id)
    if db_author:
        db.delete(db_author)
        db.commit()
        return True
    return False

# GENRES
def get_genres(db: Session):
    """Получение списка жанров."""
    return db.query(models.Genre).all()

def create_genre(db: Session, genre: schemas.GenreCreate):
    """Создание жанра."""
    db_genre = models.Genre(**genre.model_dump())
    db.add(db_genre)
    db.commit()
    db.refresh(db_genre)
    return db_genre

def delete_genre(db: Session, genre_id: int):
    """Удаление жанра."""
    genre = db.query(models.Genre).filter(models.Genre.id == genre_id).first()
    if genre:
        db.delete(genre)
        db.commit()
        return True
    return False

```

## # BOOKS

```
def get_book(db: Session, book_id: int):
    """Получение книги по ID."""
    return db.query(models.Book).filter(models.Book.id == book_id).first()

def get_books(db: Session, skip: int = 0, limit: int = 100, author_id: Optional[int] = None):
    """Получение списка книг с фильтрацией по автору."""
    query = db.query(models.Book)
    if author_id:
        query = query.filter(models.Book.author_id == author_id)
    return query.offset(skip).limit(limit).all()

def create_book(db: Session, book: schemas.BookCreate):
    """Создание книги."""
    db_book = models.Book(**book.model_dump(), available_copies=book.total_copies)
    db.add(db_book)
    db.commit()
    db.refresh(db_book)
    return db_book

def update_book(db: Session, book_id: int, book: schemas.BookCreate):
    """Обновление книги."""
    db_book = get_book(db, book_id)
    if db_book:
        for key, value in book.model_dump().items():
            setattr(db_book, key, value)
        db.commit()
        db.refresh(db_book)
    return db_book

def delete_book(db: Session, book_id: int):
    """Удаление книги."""
    db_book = get_book(db, book_id)
    if db_book:
        db.delete(db_book)
        db.commit()
        return True
    return False

# READERS

def get_reader(db: Session, reader_id: int):
    """Получение читателя по ID."""
    return db.query(models.Reader).filter(models.Reader.id == reader_id).first()
```

```

def get_readers(db: Session):
    """Получение списка читателей."""
    return db.query(models.Reader).all()

def create_reader(db: Session, reader: schemas.ReaderCreate):
    """Создание читателя."""
    db_reader = models.Reader(**reader.model_dump())
    db.add(db_reader)
    db.commit()
    db.refresh(db_reader)
    return db_reader

def update_reader(db: Session, reader_id: int, reader: schemas.ReaderCreate):
    """Обновление читателя."""
    db_reader = get_reader(db, reader_id)
    if db_reader:
        for key, value in reader.model_dump().items():
            setattr(db_reader, key, value)
        db.commit()
        db.refresh(db_reader)
    return db_reader

def delete_reader(db: Session, reader_id: int):
    """Удаление читателя."""
    db_reader = get_reader(db, reader_id)
    if db_reader:
        db.delete(db_reader)
        db.commit()
        return True
    return False

# LOANS
def create_loan(db: Session, loan: schemas.LoanCreate):
    """Создание выдачи книги."""
    book = get_book(db, loan.book_id)
    if not book or book.available_copies < 1:
        return None

    db_loan = models.Loan(
        book_id=loan.book_id,
        reader_id=loan.reader_id,
        loan_date=date.today(),
        due_date=loan.due_date,
        is_returned=False
    )

```

```

    book.available_copies -= 1
    db.add(db_loan)
    db.commit()
    db.refresh(db_loan)
    return db_loan

def get_loans(db: Session, reader_id: Optional[int] = None):
    """Получение списка выдач."""
    query = db.query(models.Loan)
    if reader_id:
        query = query.filter(models.Loan.reader_id == reader_id)
    return query.all()

def return_book(db: Session, loan_id: int):
    """Возврат книги."""
    loan = db.query(models.Loan).filter(models.Loan.id == loan_id).first()
    if not loan or loan.is_returned:
        return None

    loan.is_returned = True
    loan.return_date = date.today()

    book = get_book(db, loan.book_id)
    if book:
        book.available_copies += 1

    db.commit()
    return loan

```

## Database.py

```

"""Подключение к базе данных."""
import os

from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SESSION_LOCAL = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

```

```
def get_db():
    """Получение сессии БД."""
    db = SESSION_LOCAL()
    try:
        yield db
    finally:
        db.close()
```

## Main.py

```
"""FastAPI приложение Библиотека."""
from datetime import date
from typing import List, Optional

from fastapi import FastAPI, Depends, Form
from fastapi.responses import HTMLResponse
from sqlalchemy.orm import Session

import crud
import models
import schemas
from database import engine, get_db

# Создаем таблицы
models.Base.metadata.create_all(bind=engine)

app = FastAPI(title="Библиотека")

# ГЛАВНАЯ
@app.get("/", response_class=HTMLResponse)
def root():
    """Главная страница."""
    return """
<h1>Библиотека</h1>
<ul>
    <li><a href="/authors">Авторы</a></li>
    <li><a href="/books">Книги</a></li>
    <li><a href="/readers">Читатели</a></li>
    <li><a href="/loans">Выдача книг</a></li>
</ul>
<hr>
<h3>JSON API:</h3>
<ul>
    <li><a href="/api/authors">/api/authors</a></li>
    <li><a href="/api/books">/api/books</a></li>
    <li><a href="/api/readers">/api/readers</a></li>
    <li><a href="/api/loans">/api/loans</a></li>
</ul>
    """
```

```
"""
```

```
# АВТОРЫ
```

```
@app.get("/authors", response_class=HTMLResponse)
```

```
def authors_page(db: Session = Depends(get_db)):
```

```
    """Страница управления авторами."""
```

```
    authors = crud.get_authors(db)
```

```
    html = "<h1>Авторы</h1><a href='/'>Назад</a><br><br>"
```

```
# Форма добавления
```

```
html += """
```

```
<form method='post' action='/authors/add'>
```

```
    <input type='text' name='first_name' placeholder='Имя' required>
```

```
    <input type='text' name='last_name' placeholder='Фамилия' required>
```

```
    <input type='date' name='birth_date'>
```

```
    <input type='text' name='country' placeholder='Страна'>
```

```
    <button type='submit'>Добавить</button>
```

```
</form>
```

```
<br>
```

```
<table border='1'>
```

```
    <tr><th>ID</th><th>Имя</th><th>Фамилия</th><th>Действия</th></tr>
```

```
"""
```

```
for a in authors:
```

```
    html += f"""
```

```
    <tr>
```

```
        <td>{a.id}</td>
```

```
        <td>{a.first_name}</td>
```

```
        <td>{a.last_name}</td>
```

```
        <td>
```

```
            <form method='post' action='/authors/delete/{a.id}' style='display:inline'>
```

```
                <button type='submit'>Удалить</button>
```

```
            </form>
```

```
        </td>
```

```
    </tr>
```

```
"""
```

```
html += "</table>"
```

```
return html
```

```
@app.post("/authors/add")
```

```
def add_author( # pylint: disable=R0913,R0917
```

```
    first_name: str = Form(...),
```

```
    last_name: str = Form(...),
```

```
    birth_date: Optional[str] = Form(None),
```

```

country: Optional[str] = Form(None),
db: Session = Depends(get_db)
):
    """Добавление автора."""
    author = schemas.AuthorCreate(
        first_name=first_name,
        last_name=last_name,
        birth_date=date.fromisoformat(birth_date) if birth_date else None,
        country=country
    )
    crud.create_author(db, author)
    return authors_page(db)

```

```

@app.post("/authors/delete/{author_id}")
def delete_author(author_id: int, db: Session = Depends(get_db)):
    """Удаление автора."""
    crud.delete_author(db, author_id)
    return authors_page(db)

```

#### # КНИГИ

```

@app.get("/books", response_class=HTMLResponse)
def books_page(db: Session = Depends(get_db)):

```

```

    """Страница управления книгами."""

```

```

    books = crud.get_books(db)
    authors = crud.get_authors(db)
    genres = crud.get_genres(db)

```

```

    html = "<h1>Книги</h1><a href='/'>Назад</a><br><br>"

```

#### # Форма добавления

```

    html += "<form method='post' action='/books/add'>"
    html += "<input type='text' name='title' placeholder='Название' required>"
    html += "<select name='author_id' required><option value=''>Автор</option>"
    for a in authors:
        html += f"<option value='{a.id}'>{a.first_name} {a.last_name}</option>"
    html += "</select>"
    html += "<select name='genre_id' required><option value=''>Жанр</option>"
    for g in genres:
        html += f"<option value='{g.id}'>{g.name}</option>"
    html += "</select>"
    html += "<input type='number' name='year' placeholder='Год'>"
    html += "<input type='text' name='isbn' placeholder='ISBN'>"
    html += "<input type='number' name='total_copies' placeholder='Кол-во' value='1'>"
    html += "<input type='number' name='price' placeholder='Цена' step='0.01'>"
    html += "<button type='submit'>Добавить</button></form><br>"

```

```

html += "<table border='1'>"
html +=
"<tr><th>ID</th><th>Название</th><th>Автор</th><th>Доступно</th><th>Действия</th></tr>"
for b in books:
    html += f"""
    <tr>
        <td>{b.id}</td>
        <td>{b.title}</td>
        <td>{b.author.first_name} {b.author.last_name}</td>
        <td>{b.available_copies}/{b.total_copies}</td>
        <td>
            <form method='post' action='/books/delete/{b.id}' style='display:inline'>
                <button type='submit'>Удалить</button>
            </form>
        </td>
    </tr>
    """
html += "</table>"
return html

```

```

@app.post("/books/add")
def add_book( # pylint: disable=R0913,R0917
    title: str = Form(...),
    author_id: int = Form(...),
    genre_id: int = Form(...),
    year: Optional[int] = Form(None),
    isbn: Optional[str] = Form(None),
    total_copies: int = Form(1),
    price: float = Form(0),
    db: Session = Depends(get_db)
):
    """Добавление книги."""
    book = schemas.BookCreate(
        title=title,
        author_id=author_id,
        genre_id=genre_id,
        year=year,
        isbn=isbn,
        total_copies=total_copies,
        price=price
    )
    crud.create_book(db, book)
    return books_page(db)

```

```

@app.post("/books/delete/{book_id}")
def delete_book(book_id: int, db: Session = Depends(get_db)):

```

```

"""Удаление книги."""
crud.delete_book(db, book_id)
return books_page(db)

```

# ЧИТАТЕЛИ

```

@app.get("/readers", response_class=HTMLResponse)
def readers_page(db: Session = Depends(get_db)):
    """Страница управления читателями."""
    readers = crud.get_readers(db)

    html = "<h1>Читатели</h1><a href='/'>Назад</a><br><br>"

    html += """
    <form method='post' action='/readers/add'>
        <input type='text' name='first_name' placeholder='Имя' required>
        <input type='text' name='last_name' placeholder='Фамилия' required>
        <input type='email' name='email' placeholder='Email' required>
        <input type='text' name='phone' placeholder='Телефон'>
        <input type='text' name='address' placeholder='Адрес'>
        <button type='submit'>Добавить</button>
    </form>
    <br>
    <table border='1'>
        <tr><th>ID</th><th>Имя</th><th>Фамилия</th><th>Email</th><th>Действия</th></tr>
    """

    for r in readers:
        html += f"""
        <tr>
            <td>{r.id}</td>
            <td>{r.first_name}</td>
            <td>{r.last_name}</td>
            <td>{r.email}</td>
            <td>
                <form method='post' action='/readers/delete/{r.id}' style='display:inline'>
                    <button type='submit'>Удалить</button>
                </form>
            </td>
        </tr>
    """

    html += "</table>"
    return html

```

```

@app.post("/readers/add")
def add_reader( # pylint: disable=R0913,R0917

```

```

first_name: str = Form(...),
last_name: str = Form(...),
email: str = Form(...),
phone: Optional[str] = Form(None),
address: Optional[str] = Form(None),
db: Session = Depends(get_db)
):

```

```

    """Добавление читателя."""

```

```

    reader = schemas.ReaderCreate(
        first_name=first_name,
        last_name=last_name,
        email=email,
        phone=phone,
        address=address
    )

```

```

    crud.create_reader(db, reader)
    return readers_page(db)

```

```

@app.post("/readers/delete/{reader_id}")

```

```

def delete_reader(reader_id: int, db: Session = Depends(get_db)):

```

```

    """Удаление читателя."""

```

```

    crud.delete_reader(db, reader_id)
    return readers_page(db)

```

```

# ВЫДАЧА КНИГ

```

```

@app.get("/loans", response_class=HTMLResponse)

```

```

def loans_page(db: Session = Depends(get_db)):

```

```

    """Страница управления выдачей книг."""

```

```

    loans = crud.get_loans(db)
    books = crud.get_books(db)
    readers = crud.get_readers(db)

```

```

    html = "<h1>Выдача книг</h1><a href='/'>Назад</a><br><br>"

```

```

    html += "<form method='post' action='/loans/add'>"

```

```

    html += "<select name='book_id' required><option value=''>Книга</option>"

```

```

    for b in books:

```

```

        text = f"{b.title} (доступно: {b.available_copies})"

```

```

        html += f"<option value='{b.id}'>{text}</option>"

```

```

    html += "</select>"

```

```

    html += "<select name='reader_id' required><option value=''>Читатель</option>"

```

```

    for r in readers:

```

```

        html += f"<option value='{r.id}'>{r.first_name} {r.last_name}</option>"

```

```

    html += "</select>"

```

```

    html += "<input type='date' name='due_date' required>"

```

```

    html += "<button type='submit'>Выдать</button></form><br>"

```

```

html += "<table border='1'>"
html +=
"<tr><th>ID</th><th>Книга</th><th>Читатель</th><th>Статус</th><th>Действия</th></tr>"
for loan in loans:
    status = "Возвращена" if loan.is_returned else "На руках"
    html += f"""
    <tr>
        <td>{loan.id}</td>
        <td>{loan.book.title}</td>
        <td>{loan.reader.first_name} {loan.reader.last_name}</td>
        <td>{status}</td>
        <td>
            """
    if not loan.is_returned:
        html += f"""
            <form method='post' action='/loans/return/{loan.id}' style='display:inline'>
                <button type='submit'>Вернуть</button>
            </form>
            """
    html += "<tr></tr>"

html += "</table>"
return html

```

```

@app.post("/loans/add")
def add_loan(
    book_id: int = Form(...),
    reader_id: int = Form(...),
    due_date: str = Form(...),
    db: Session = Depends(get_db)
):
    """Добавление выдачи книги."""
    loan = schemas.LoanCreate(
        book_id=book_id,
        reader_id=reader_id,
        due_date=date.fromisoformat(due_date)
    )
    crud.create_loan(db, loan)
    return loans_page(db)

```

```

@app.post("/loans/return/{loan_id}")
def return_loan(loan_id: int, db: Session = Depends(get_db)):
    """Возврат книги."""
    crud.return_book(db, loan_id)
    return loans_page(db)

```

```

# JSON API
@app.get("/api/authors", response_model=List[schemas.AuthorResponse])
def api_authors(db: Session = Depends(get_db)):
    """API: список авторов."""
    return crud.get_authors(db)

@app.get("/api/books", response_model=List[schemas.BookResponse])
def api_books(db: Session = Depends(get_db)):
    """API: список книг."""
    return crud.get_books(db)

@app.get("/api/readers", response_model=List[schemas.ReaderResponse])
def api_readers(db: Session = Depends(get_db)):
    """API: список читателей."""
    return crud.get_readers(db)

@app.get("/api/loans", response_model=List[schemas.LoanResponse])
def api_loans(db: Session = Depends(get_db)):
    """API: список выдач."""
    return crud.get_loans(db)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)

```

## Models.py

```

"""Модели базы данных (5 таблиц)."""
from datetime import date
from sqlalchemy import Column, Integer, String, Date, ForeignKey, Numeric, Boolean
from sqlalchemy.orm import relationship
from database import Base

class Author(Base): # pylint: disable=R0903
    """Авторы."""
    __tablename__ = "authors"

    id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    birth_date = Column(Date, nullable=True)
    country = Column(String(100), nullable=True)

    books = relationship("Book", back_populates="author")

```

```

class Genre(Base): # pylint: disable=R0903
    """Жанры."""
    __tablename__ = "genres"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False, unique=True)
    description = Column(String(500), nullable=True)

    books = relationship("Book", back_populates="genre")

class Book(Base): # pylint: disable=R0903
    """Книги."""
    __tablename__ = "books"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    author_id = Column(Integer, ForeignKey("authors.id"), nullable=False)
    genre_id = Column(Integer, ForeignKey("genres.id"), nullable=False)
    year = Column(Integer, nullable=True)
    isbn = Column(String(20), unique=True, nullable=True)
    total_copies = Column(Integer, default=1)
    available_copies = Column(Integer, default=1)
    price = Column(Numeric(10, 2), default=0)

    author = relationship("Author", back_populates="books")
    genre = relationship("Genre", back_populates="books")
    loans = relationship("Loan", back_populates="book")

class Reader(Base): # pylint: disable=R0903
    """Читатели."""
    __tablename__ = "readers"

    id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    email = Column(String(200), unique=True, nullable=False)
    phone = Column(String(20), nullable=True)
    address = Column(String(500), nullable=True)
    registration_date = Column(Date, default=date.today)

    loans = relationship("Loan", back_populates="reader")

class Loan(Base): # pylint: disable=R0903
    """Выдача книг."""

```

```

__tablename__ = "loans"

id = Column(Integer, primary_key=True, index=True)
book_id = Column(Integer, ForeignKey("books.id"), nullable=False)
reader_id = Column(Integer, ForeignKey("readers.id"), nullable=False)
loan_date = Column(Date, default=date.today, nullable=False)
due_date = Column(Date, nullable=False)
return_date = Column(Date, nullable=True)
is_returned = Column(Boolean, default=False)

book = relationship("Book", back_populates="loans")
reader = relationship("Reader", back_populates="loans")

```

## Schemas.py

```

"""Pydantic схемы для валидации."""
from datetime import date
from typing import Optional

from pydantic import BaseModel

class AuthorCreate(BaseModel):
    """Схема создания автора."""
    first_name: str
    last_name: str
    birth_date: Optional[date] = None
    country: Optional[str] = None

class AuthorResponse(AuthorCreate):
    """Схема ответа с автором."""
    id: int

    class Config: # pylint: disable=R0903
        """Настройки схемы."""
        from_attributes = True

class GenreCreate(BaseModel):
    """Схема создания жанра."""
    name: str
    description: Optional[str] = None

class GenreResponse(GenreCreate):
    """Схема ответа с жанром."""
    id: int

    class Config: # pylint: disable=R0903
        """Настройки схемы."""

```

```
from_attributes = True
```

```
class BookCreate(BaseModel):  
    """Схема создания книги."""  
    title: str  
    author_id: int  
    genre_id: int  
    year: Optional[int] = None  
    isbn: Optional[str] = None  
    total_copies: int = 1  
    price: float = 0
```

```
class BookResponse(BookCreate):  
    """Схема ответа с книгой."""  
    id: int  
    available_copies: int  
  
    class Config: # pylint: disable=R0903  
        """Настройки схемы."""  
        from_attributes = True
```

```
class ReaderCreate(BaseModel):  
    """Схема создания читателя."""  
    first_name: str  
    last_name: str  
    email: str  
    phone: Optional[str] = None  
    address: Optional[str] = None
```

```
class ReaderResponse(ReaderCreate):  
    """Схема ответа с читателем."""  
    id: int  
    registration_date: date  
  
    class Config: # pylint: disable=R0903  
        """Настройки схемы."""  
        from_attributes = True
```

```
class LoanCreate(BaseModel):  
    """Схема создания выдачи книги."""  
    book_id: int  
    reader_id: int  
    due_date: date
```

```
class LoanResponse(BaseModel):
    """Схема ответа с выдачей книги."""
    id: int
    book_id: int
    reader_id: int
    loan_date: date
    due_date: date
    return_date: Optional[date]
    is_returned: bool

    class Config: # pylint: disable=R0903
        """Настройки схемы."""
        from_attributes = True
```

Для запуска необходимо использовать `python main.py` затем перейти по ссылке <http://127.0.0.1:8000/docs>

Результат выполнения :

# Библиотека

- [Авторы](#)
- [Книги](#)
- [Читатели](#)
- [Выдача книг](#)

# Авторы

[Назад](#)



Добавить

| ID | Имя       | Фамилия     | Действия           |
|----|-----------|-------------|--------------------|
| 1  | Лев       | Толстой     | <div>Удалить</div> |
| 2  | Федор     | Достоевский | <div>Удалить</div> |
| 3  | Джордж    | Оруэлл      | <div>Удалить</div> |
| 4  | Михаил    | Булгаков    | <div>Удалить</div> |
| 5  | Александр | Пушкин      | <div>Удалить</div> |
| 6  | ds        | adsf        | <div>Удалить</div> |
| 7  | ds        | adsf        | <div>Удалить</div> |
| 8  | ds        | adsf        | <div>Удалить</div> |

# Читатели

[Назад](#)

Добавить

| ID | Имя     | Фамилия | Email          | Действия           |
|----|---------|---------|----------------|--------------------|
| 1  | Иван    | Петров  | ivan@mail.ru   | <div>Удалить</div> |
| 2  | Мария   | Иванова | maria@mail.ru  | <div>Удалить</div> |
| 3  | Алексей | Сидоров | alexey@mail.ru | <div>Удалить</div> |
| 4  | Елена   | Козлова | elena@mail.ru  | <div>Удалить</div> |
| 5  | выф     | ы       | fafa@mail.ru   | <div>Удалить</div> |

Вариант 9. База данных Справочное бюро ж/д вокзала  
Таблицы в БД:

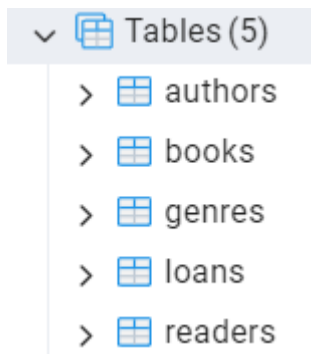
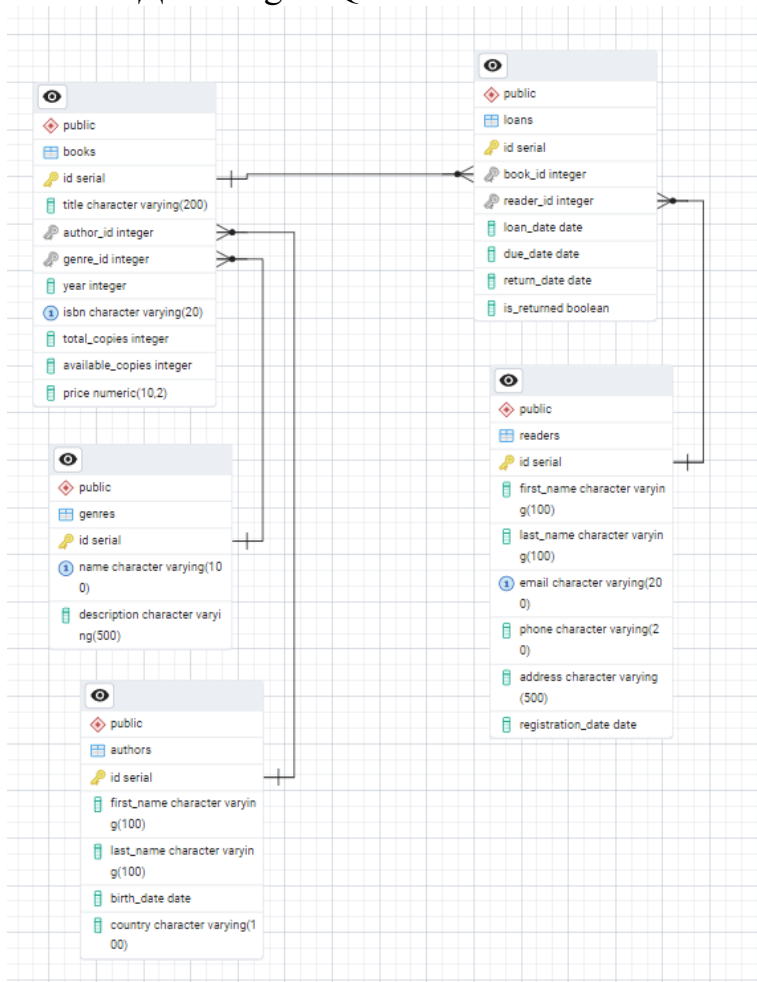


Схема БД в PostgreSQL:



Вывод: приобрел практические навыки разработки API и баз данных